Spis treści

1. LabZSK – informacje podstawowe 2
2. Rejestry procesora 4
3. Lista rozkazów komputera LabZSK 5
   3.1. Cykl rozkazowy 11
   3.2. Formaty rozkazów 12
   3.3. Metody adresacji 13
4. Budowa jednostki centralnej 14
5. Sterowanie mikroprogramowe 17
   5.1. Organizacja pamięci stałej 17
   5.2. Sygnały sterujące 19
   5.3. Cykle pracy pamięci operacyjnej 24
6. Graficzny interfejs symulatora 25
7. Przykładowy mikroprogram pobrania rozkazu 33
8. Rozpoczęcie pobrania rozkazu w symulatorze LabZSK 39
Wykaz tabel 43
Wykaz rysunków 44

1. LabZSK – informacje podstawowe

Komputer LabZSK, bazujący na architekturze jednoadresowej z 16-bitowym słowem, zbudowany jest z jednostki centralnej połączonej z pamięcią operacyjną za pomocą magistrali systemowej. Jednostka centralna, ze względu na realizowaną obsługę pamięci operacyjnej, zbudowana jest z połączonych szyną: sumatora arytmetyczno-logicznego ALU, zestawu rejestrów oraz jednostki sterującej.

Pamięć główna komputera LabZSK może pracować w trzech cyklach: odczyt-regeneracja, zerowanie-zapis albo przerwaniowy; udostępniając 256 16-bitowych komórek, służących do przechowywania danych, adresów i rozkazów.

Sterowanie w komputerze LabZSK realizowane jest za pomocą mikroprogramów umieszczonych w pamięci stałej. Pamięć mikroprogramów, znajdująca się w jednostce sterującej, może pomieścić 256 48-bitowych mikrorozkazów. Każdy pobrany mikrorozkaz dzieli się na 10 mikrooperacji oraz adres następnego mikrorozkazu do wykonania, przy spełnionym warunku skoku.

Zawartość pamięci stałej, czyli mikroprogramy, służy do realizacji rozkazów umieszczanych w komórkach pamięci operacyjnej. Do dyspozycji programisty oddano 31 rozkazów zwykłych, 16 rozszerzonych oraz zapewniono kilka trybów adresacji (które określają 3-bity rozkazu). Z racji zastosowania programowalnej jednostki sterującej, w pamięci sterującej muszą znajdować się mikroprogramy interpretujące wszystkie 47 dostępnych rozkazów. Dodatkowo dostępna jest adresacja względna i pośrednia, rozbudowana lista rozkazów oraz niewielka liczba rejestrów, jak w modelowym komputerze CISC.

Cykl rozkazowy komputera LabZSK możemy podzielić na: pobranie rozkazu, adresowanie pośrednie, wykonanie rozkazu i obsługa przerwań. Każdy cykl jest wykonywany jako mikroprogram zawarty w pamięci stałej, przy czym zawartość pamięci sterującej musi uwzględniać, że cykle pobrania i wykonania występują zawsze dla każdego rozkazu, a adresowanie pośrednie i obsługa przerwań są opcjonalne.

Cykl mikrorozkazu, składający się z podcykli pobrania oraz wykonania, jest podstawowym zdarzeniem w mikroprogramowanym procesorze. Pobranie mikrorozkazu realizowane jest jako przesłanie zawartości komórki pamięci stałej, wskazywanej przez RAPS, do rejestru buforowego pamięci stałej. Wykonanie mikrorozkazu polega na realizacji zawartych w nim mikrooperacji, w kolejnych taktach pracy jednostki sterującej. Mikrooperacje, składające się na mikrorozkaz, są w rzeczywistości sygnałami sterującymi, przesyłanymi do wnętrza jednostki sterującej, celem realizacji elementarnych operacji.

Symulator komputera LabZSK umożliwia tworzenie, modyfikowanie oraz wykonywanie programów znajdujących się w pamięci operacyjnej. Każdy rozkaz wykonywany jest zgodnie z mikroprogramem, umieszczonym w odpowiednim obszarze pamięci stałej. Co więcej zawartość pamięci mikroprogramów też może być ustalana przez użytkownika. Dodatkowo dla obu elementów dostępna jest opcja zapisu zawartości do pliku, celem umożliwienia jej ponownego wykorzystania.

Uruchomienie symulatora LabZSK wymaga zainstalowanego środowiska .NET w wersji 4.0 lub nowszej. Ponadto praca z symulatorem wymaga pliku konfiguracyjnego, zawierającego informacje o funkcjach jakie symulator udostępnia operatorowi. Brak pliku konfiguracyjnego będzie powodował stworzenie i załadowanie jego podstawowej wersji (trybu studenta), ograniczającej np. możliwość wykorzystania trybu automatycznej symulacji.

2. Rejestry procesora

Rejestry procesora stanowią najwyższy szczebel w hierarchii pamięci, będąc najszybszym (w rzeczywistych maszynach) dostępnym rodzajem pamięci komputerowej. Ich głównym zadaniem jest realizacja chwilowego przechowywania danych, adresów i rozkazów w procesorze. W strukturze procesora, rejestry znajdują się wewnątrz jednostki centralnej i komunikują się z pozostałymi elementami za pomocą wewnętrznej magistrali.

Komputer LabZSK posiada trzynaście rejestrów o odpowiednio dostosowanej pojemności, przeznaczonych do przechowywania instrukcji, danych i adresów. Pełna lista rejestrów została przedstawiona w tabeli 2.1.

Mnemonik	Nazwa rejestru	Pojemność [bit]	Dodatkowy opis
A	Akumulator	16	Przechowuje wyniki operacji
LK	Licznik kroków	7	Zawiera liczbę przesunięć dla rozkazów przesunięć lub liczbę iteracji mnożenia/dzielenia
LR	Licznik rozkazów	16	Zawiera adres następnego rozkazu do wykonania
MQ	Rejestr iloczynu/ilorazu	16	Zawiera operand iloczynu/ilorazu
RAE	Rejestr adresu efektywnego	16	Zawiera adres efektywny operandu
RAP	Rejestr adresowy pamięci	8	Przechowuje adres komórki PAO dla przesłań do/z PAO
RAPS	Rejestr adresowy pamięci stałej	8	Zawiera adres następnego mikrorozkazu do wykonania
RBP	Rejestr buforowy pamięci	16	Zawiera daną przesłaną do/z PAO
RBPS	Rejestr buforowy pamięci stałej	48	Zawiera aktualnie wykonywany mikrorozkaz
RI	Rejestr indeksowy	16	Przechowuje wartość modyfikującą adres rozkazu
RR	Rejestr rozkazów	16	Zawiera aktualnie wykonywany rozkaz
SUMA	Rejestr sumy adresowej	8	Zawiera wynik sumatora adresowego
X	Rejestr danych	16	Rejestr pomocniczy

Tabela 2.1. Rejestry komputera LabZSK (Alfabetycznie).

3. Lista rozkazów komputera LabZSK

Każdy rozkaz przechowywany w pamięci operacyjnej stanowi pojedynczą instrukcję dla jednostki centralnej. Podczas pracy komputera, pobrany z pamięci operacyjnej rozkaz przesyłany jest do rejestru RR procesora, dekodowany, a następnie wykonywany z wykorzystaniem jednostki sterującej.

W komputerze LabZSK rozkazy możemy podzielić ze względu na format rozkazu:

- zwykły (zobrazowane w tabeli 3.1 oraz tabeli 3.2),
- rozszerzony (przedstawione w tabela 3.3).

Wartość dziesiętna	Kod binarny	Mnemonik	Pełna nazwa	Definicja rozkazu
1	00001	ADD	Dodawanie	A+PAO[RAE] → A
2	00010	SUB	Odejmowanie	A-PAO[RAE] → A
3	00011	MUL	Mnożenie	A*PAO[RAE] → A‖MQ
4	00100	DIV	Dzielenie	A/PAO[RAE] → MQ reszta w A
5	00101	STQ	Zapamiętaj rejestr MQ	MQ→ PAO[RAE]
6	00110	STA	Zapamiętaj akumulator A	A → PAO[RAE]
7	00111	STX	Zapamiętaj modyfikator RI	RI → PAO[RAE]
8	01000	LDA	Ładuj A	PAO[RAE] → A
9	01001	LDX	Ładuj modyfikator RI	PAO[RAE] → RI
10	01010	STC	Zapamiętaj LR	LR → PAO[RAE]
11	01011	TXA	Prześlij RI do A	RI → A
12	01100	TMQ	Prześlij MQ do A	MQ → A
13	01101	ADX	Dodaj do RI	RI+PAO[RAE] → RI
14	01101	SIO	Start operacji WE-WY	PAO[RAE] → IOAR
15	01111	LIO	Ładuj licznik słów WE-WY	PAO[RAE] → IOWC
16	10000	UNB	Skok bezwarunkowy	RAE → LR
17	10001	BAO	Skocz jeśli nadmiar A	jeśli OFF=1; RAE →LR

Tabela 3.1. Rozkazy komputera LabZSK w formacie zwykłym – cz. 1.

Wartość dziesiętna	Kod binarny	Mnemonik	Pełna nazwa	Definicja rozkazu
18	10010	BXP	Skocz jeśli RI dodatni	jeśli RI>0; RAE→LR
19	10011	BXZ	Skocz jeśli RI=0	jeśli RI=0; RAE→LR
20	10100	BXN	Skocz jeśli RI ujemny	jeśli RI<0; RAE→LR
21	10101	TLD	RI=RI-1 i skocz jeśli RI>0	Jeśli RI>0; to RI-1→RI; RAE→LR
22	10110	BAP	Skocz jeśli A dodatni	jeśli A>0; RAE→LR
23	10111	BAZ	Skocz jeśli A=0	jeśli A=0; RAE→LR
24	11000	BAN	Skocz jeśli A ujemny	jeśli A<0; RAE→LR
25	11001	LOR	Suma logiczna A i komórki	A OR PAO[RAE] → A
26	11010	LPR	Iloczyn logiczny A i komórki	A AND PAO[RAE] → A
27	11011	LNG	Negacja logiczna A	NOT A → A
28	11100	EOR	Różnica symetryczna A i komórki	A XOR PAO[RAE] → A
29	11101	SRJ	Skok ze śladem	LR→PAO[RAE] RAE+1→ LR
30	11110	BDN	Skocz jeśli urządzenie niedostępne	jeśli urządzenie niedostępne PAO[RAE]→LR
31	11111	NOP	Nic nie rób	skok do mikroprogramu pobrania rozkazu

Tabela 3.2. Rozkazy komputera LabZSK w formacie zwykłym – cz. 2.

Wartość dziesiętna	Kod binarny	Mnemonik	Pełna nazwa	Definicja rozkazu
0	0000	STP	Stop dynamiczny	Skok na siebie
1	0001	CMA	Uzupełnienie A do 2	-A→A
2	0010	ALA	Przesunięcie arytmetyczne w lewo A	R(A)→L (A); $0 \rightarrow A_{15}$;N-razy
3	0011	ARA	Przesunięcie arytmetyczne w prawo A	L(A)→R(A); $A_0 \rightarrow A_1$; N-razy
4	0100	LRQ	Przesunięcie logiczne w prawo A||MQ	R(A||MQ);$0 \rightarrow A_0$, N-razy
5	0101	LLQ	Przesunięcie logiczne w lewo A||MQ	L(A||MQ);$0 \rightarrow MQ_{15}$ N-razy
6	0110	LLA	Przesunięcie logiczne w lewo A	R(A)→L(A),$0 \rightarrow A_{15}$, N-razy
7	0111	LRA	Przesunięcie logiczne w prawo A	L(A)→R(A), $0 \rightarrow A_0$, N-razy
8	1000	LCA	Przesunięcie cykliczne w lewo A	R(A) →L(A); $A_0 \rightarrow A_{15}$, N-razy
9	1001	LAI	Ładuj A bezpośrednio	N → A
10	1010	LXI	Ładuj RI bezpośrednio	N → RI
11	1011	INX	Zwiększ modyfikator RI	RI+N → RI
12	1100	DEX	Zmniejsz modyfikator RI	RI-N → RI
13	1101	CND	Podłącz urządzenie	$RR_{9\text{-}15} \rightarrow IOCR_{9\text{-}15}$
14	1101	ENI	Zezwolenie na przerwania	
15	1111	LDS	Podaj status	$IOCR_{0\text{-}11} \rightarrow A_{4\text{-}15}$, $0 \rightarrow A_{0\text{-}3}$

Tabela 3.3. Rozkazy w formacie rozszerzonym.

Alternatywnie rozkazy można podzielić ze względu na realizowaną funkcję:

- arytmetyczne,
- logiczne,
- przesłań,
- skoku,
- przesunięć.

Wartość dziesiętna	Kod binarny	Mnemonik	Pełna nazwa
Rozkazy w formacie zwykłym			
1	00001	ADD	Dodawanie
2	00010	SUB	Odejmowanie
3	00011	MUL	Mnożenie
4	00100	DIV	Dzielenie
13	01101	ADX	Dodaj do RI
Rozkazy w formacie rozszerzonym			
1	0001	CMA	Uzupełnienie A do 2
11	1011	INX	Zwiększ modyfikator RI
12	1100	DEX	Zmniejsz modyfikator RI

Tabela 3.4. Rozkazy arytmetyczne komputera LabZSK.

Wartość dziesiętna	Kod binarny	Mnemonik	Pełna nazwa
Rozkazy w formacie zwykłym			
25	11001	LOR	Suma logiczna A i komórki
26	11010	LAND	Iloczyn logiczny A i komórki
27	11011	LNG	Negacja logiczna A
28	11100	EOR	Różnica symetryczna A i komórki

Tabela 3.5. Rozkazy logiczne komputera LabZSK.

Wartość dziesiętna	Kod binarny	Mnemonik	Pełna nazwa
Rozkazy w formacie zwykłym			
5	00101	STQ	Zapamiętaj rejestr MQ
6	00110	STA	Zapamiętaj akumulator A
7	00111	STX	Zapamiętaj modyfikator RI
8	01000	LDA	Ładuj A
9	01001	LDX	Ładuj modyfikator RI
10	01010	STC	Zapamiętaj LR
11	01011	TXA	Prześlij RI do A
12	01100	TMQ	Prześlij MQ do A
Rozkazy w formacie rozszerzonym			
9	1001	LAI	Ładuj A bezpośrednio
10	1010	LXI	Ładuj RI bezpośrednio

Tabela 3.6. Rozkazy przesłań komputera LabZSK.

Wartość dziesiętna	Kod binarny	Mnemonik	Pełna nazwa
Rozkazy w formacie zwykłym			
16	10000	UNB	Skok bezwarunkowy
17	10001	BAO	Skocz jeśli nadmiar A
18	10010	BXP	Skocz jeśli RI dodatni
19	10011	BXZ	Skocz jeśli RI=0
20	10100	BXN	Skocz jeśli RI ujemny
21	10101	TLD	RI=RI-1 i skocz jeśli RI>0
22	10110	BAP	Skocz jeśli A dodatni
23	10111	BAZ	Skocz jeśli A=0
24	11000	BAN	Skocz jeśli A ujemny
29	11101	SRJ	Skok ze śladem
30	11110	BDN	Skocz jeśli urządzenie niedostępne

Tabela 3.7. Rozkazy skoku komputera LabZSK.

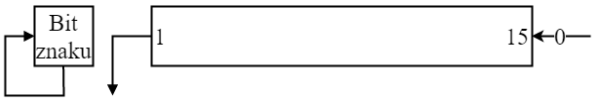
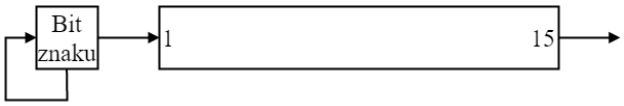
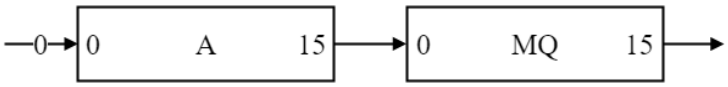
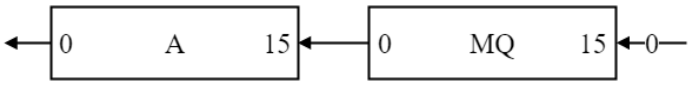
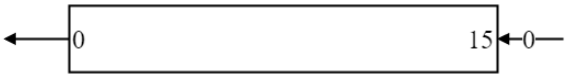
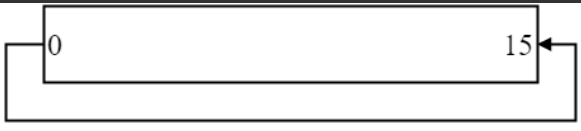
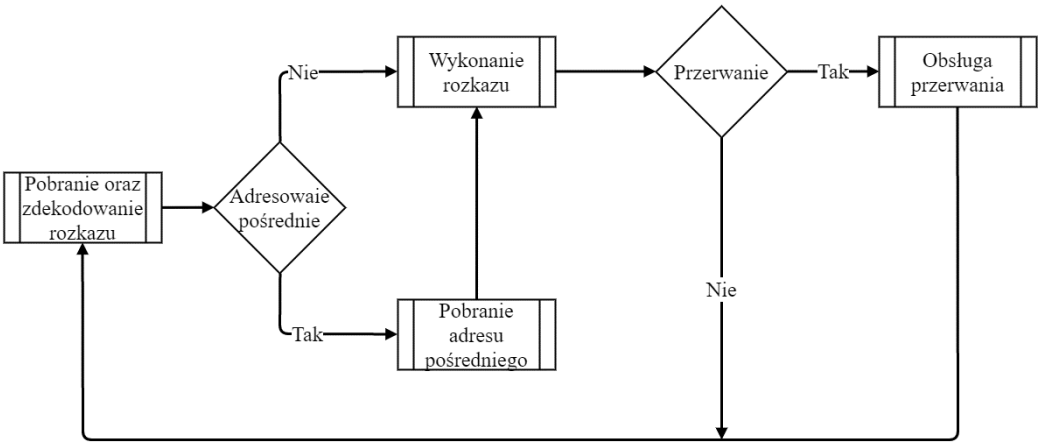
Wartość dziesiętna	Mnemonik	Pełna nazwa	Dodatkowy opis
Rozkazy w formacie rozszerzonym			
2	ALA	Przesunięcie arytmetyczne w lewo A	Bit znaku nie jest nadpisywany podczas przesunięcia, najmłodszy bit jest uzupełniany zerem
			
3	ARA	Przesunięcie arytmetyczne w prawo A	Bit znaku jest powielany, najmłodszy bit jest tracony
			
4	LRQ	Przesunięcie logiczne w prawo A||MQ	Przesunięcie złożonych rejestrów A i MQ w prawo
			
5	LLQ	Przesunięcie logiczne w lewo A||MQ	Przesunięcie złożonych rejestrów A i MQ w lewo
			
6	LLA	Przesunięcie logiczne w lewo A	Bit znaku jest tracony, najmłodszy bit jest uzupełniany zerem
			
7	LRA	Przesunięcie logiczne w prawo A	Najmłodszy bit jest tracony, bit znaku jest uzupełniany zerem
			
8	LCA	Przesunięcie cykliczne w lewo A	Aktualny bit znaku zostaje przesunięty na pozycję najmłodszego bitu
			

Tabela 3.8. Rozkazy przesunięć komputera LabZSK.

3.1. Cykl rozkazowy

W komputerze LabZSK cykl rozkazowy można podzielić na cztery etapy[1], widoczne na rys. 3.1. Na etapie pobrania, następuje załadowanie rozkazu z pamięci do rejestru RR oraz ustalenie czy występuje adresowanie pośrednie. Jeśli rozkaz określa adresowanie pośrednie, to cykl wykonania musi być poprzedzony cyklem adresowania pośredniego. W cyklu adresowania pośredniego, pole adresowe rozkazu zostaje załadowane do rejestru RAE, celem pobrania argumentu wskazywanego przez ten adres. Po wykonaniu rozkazu, sprawdzane jest wystąpienie przerwania, jeśli takowe wystąpiło to następuje obsługa przerwania. Przed obsługą przerwania, aktualna pozycja w programie (wartość rejestru LR) jest zapamiętywana, aby po wykonaniu mikroprogramu obsługi przerwań można było wrócić do dalszego wykonywania programu. W ramach obsługi przerwań, występuje załadowanie adresu podprogramu obsługi przerwań, jego wykonanie, pobranie z pamięci zachowanej pozycji w programie i wykonanie kolejnego cyklu rozkazowego.

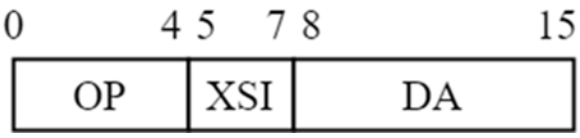


Rys. 3.1. Algorytm cyklu rozkazowy.

[1] W przykładowych mikroprogramach dla komputera LabZSK, cykl rozkazowy rozpoczynany jest od sprawdzenia wystąpienia przerwania.

3.2. Formaty rozkazów

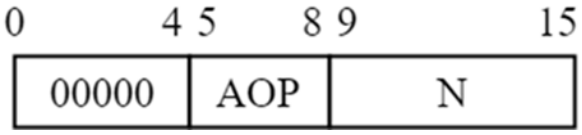
W komputerze LabZSK rozkazy w formacie zwykłym, zgodnie z długością słowa, zajmują 16 bitów oraz składają się z trzech pól widocznych na rys. 3.2. Pięć najbardziej znaczących bitów stanowi kod operacji w formacie zwykłym, kolejne trzy bity określają tryb adresacji (rozkazu zwykłego), osiem najmniej znaczących bitów stanowi część adresową.



Rys. 3.2. Format rozkazu zwykłego komputera LabZSK.

Oznaczenia: OP – część operacyjna, XSI – część modyfikacji adresu, DA – pole adresowe

Dla rozkazów w formacie rozszerzonym, widocznym na rys. 3.3, nie występuje pole modyfikujące adresację. Pięć najstarszych bitów zawsze pozostaje wyzerowane a następujące po nich cztery bity stanowią część operacyjną. Siedem najmłodszych bitów tworzy ostatnie pole w formacie rozszerzonym i jest operandem (dodatnia liczba binarna).



Rys. 3.3. Format rozkazu rozszerzonego komputera LabZSK.

Oznaczenia: OP – wyzerowane pole kodu operacji (sygnalizujące format rozszerzony), AOP – część operacyjna, N – pole operandu

3.3. Metody adresacji

Dla rozkazów w formacie zwykłym, bity (flagi) XSI wskazują jaką metodę adresacji, z tabeli 3.8, należy zastosować podczas wyznaczania adresu efektywnego operandu. Ustawienie bitu X odpowiada za zmodyfikowanie adresu o zawartość rejestru indeksowego RI. Ustawienie bitu S skutkuje zmianą adresacji o aktualną zawartość licznika rozkazów LR. Ustawienie bitu I odpowiada za ustawienie adresacji pośredniej. Ustawienie jednocześnie bitów X, S jest niedozwolone i skutkuje załadowaniem rejestru adresu efektywnego zerem. Dla rozkazu w którym wszystkie bity trybu adresacji są ustawione, rejestr adresu efektywnego zostanie załadowany zerem a jego wartość posłuży do pobrania argumentu w cyklu adresacji pośredniej.

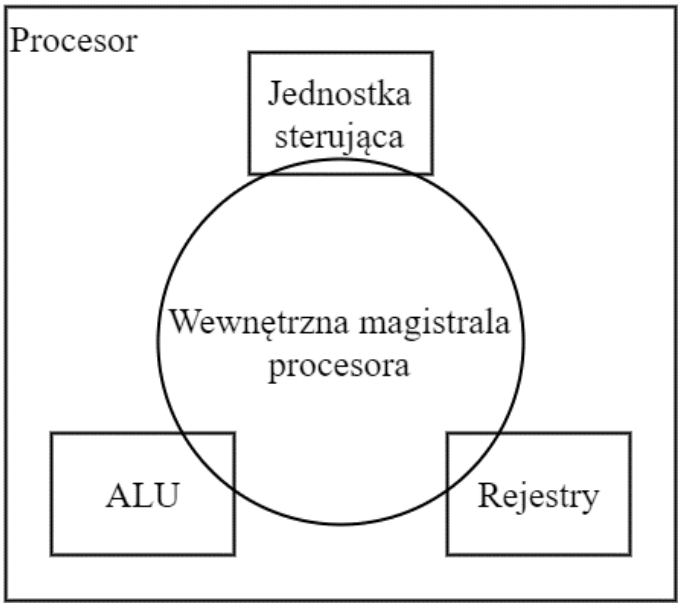
Metoda adresacji	X	S	I	Wzór na adres operandu	Sumator adresowy
Bezpośrednia	0	0	0	EA[2]=DA	L=DA, R=0
Pośrednia	0	0	1	EA=PAO[DA]	
Względna	0	1	0	EA=DA + LR	L=DA, R=LR
Pośrednia względna	0	1	1	EA=PAO[DA + LR]	
Indeksowa	1	0	0	EA=DA + RI	L=DA, R=RI
Pośrednia indeksowa	1	0	1	EA=PAO[DA + RI]	
Niedozwolona	1	1	0	EA=0	L=R=0
Niedozwolona (pośrednia)	1	1	1	EA=PAO[0]	

Tabela 3.9. Metody adresacji komputera LabZSK.

[2] EA (ang. *Effective Addresses*) – Adres efektywny

4. Budowa jednostki centralnej

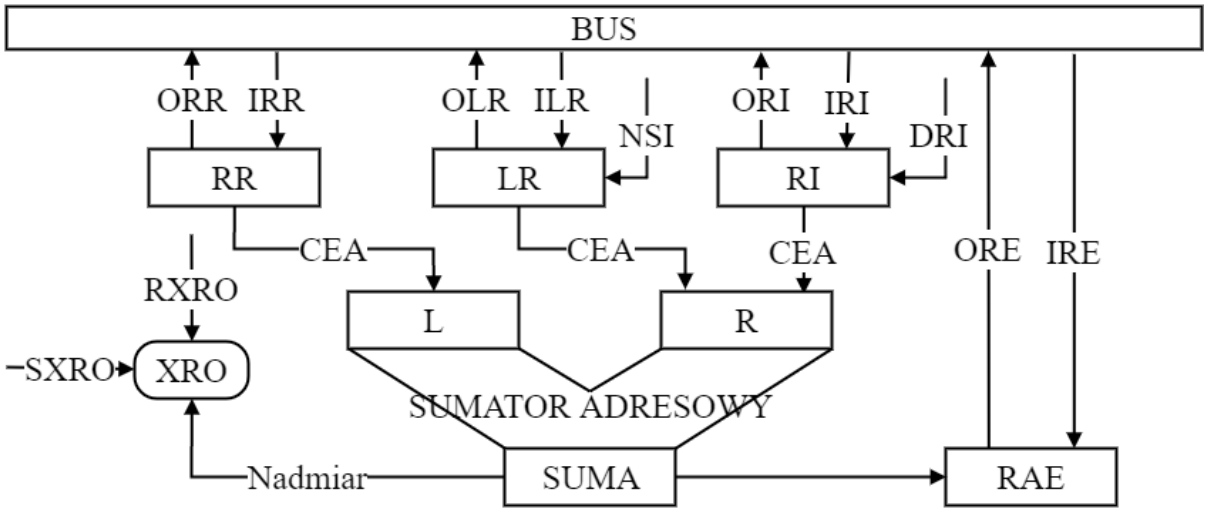
Rejestry procesora stanowią najwyższy szczebel w hierarchii pamięci, będąc najszybszym (w rzeczywistych maszynach) dostępnym rodzajem pamięci komputerowej. Ich głównym zadaniem jest realizacja chwilowego przechowywania danych, adresów i rozkazów w procesorze. W strukturze procesora, jak na rys. 4.1, rejestry znajdują się wewnątrz jednostki centralnej i komunikują się z pozostałymi elementami za pomocą wewnętrznej magistrali.



Rys. 4.1. Budowa jednostki centralnej.

Metody adresacji, przedstawione w tabeli 3.8, służą do modyfikowania adresu efektywnego operandu. Wyznaczeniem tego adresu zajmuje się sumator adresowy. Wchodzi on w skład bloku pobrania rozkazu, stanowiącego element logicznego podziału jednostki centralnej. Blok pobrania rozkazu wyznacza adres efektywny operandu, zgodnie z metodą adresacji określną przez bity XSI, danego rozkazu zwykłego. Ponieważ wyznaczanie adresu efektywnego polega na sumowaniu wartości, w bloku pobrania rozkazu występuje, równolegle do ALU, sumator adresowy.

Rejestry, sumator adresowy oraz połącznia[3] między elementami w bloku pobrania rozkazu, zostały przedstawione na rys. 4.2.



Rys. 4.2. Blok pobrania rozkazu komputera LabZSK.

Przepływy oznaczone mnemonikiem CEA należy interpretować zgodnie z zawartościami z tabeli 3.8.

Flaga XRO jest zapalana przez wyjście sumatora adresowego, jeśli wykryty został nadmiar (przekroczono wartość 255) lub bity XS rozkazu[4] są jednocześnie ustawione.

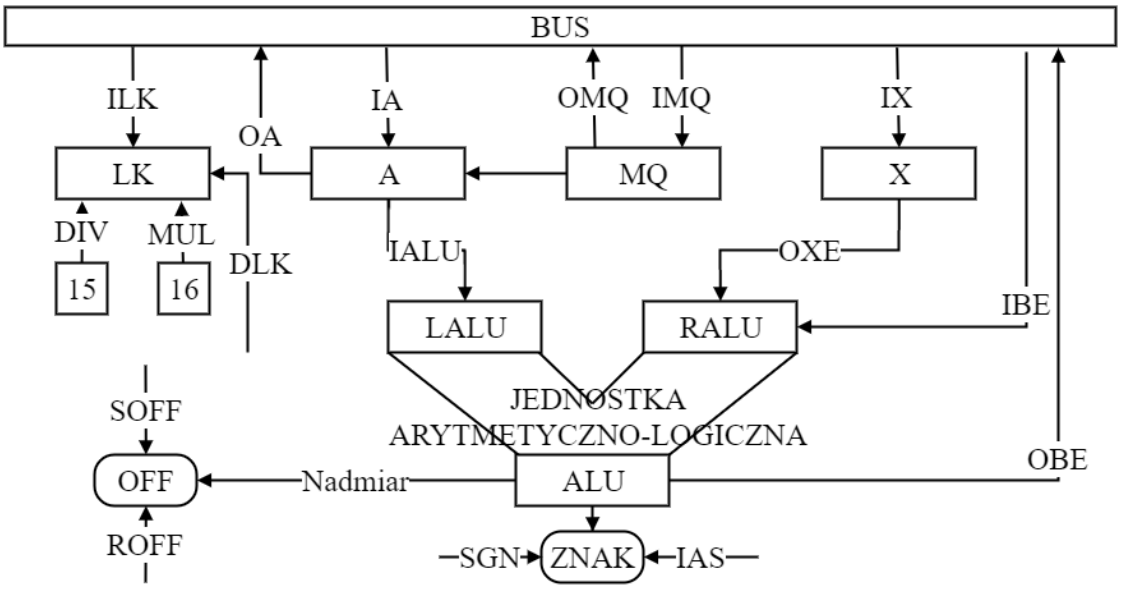
---

[3] Połączenia przedstawione na schemacie opisane są mnemonikami mikrooperacji przesłań komputera LabZSK.

[4] Aktualnie wykonywany rozkaz jest przechowywany w rejestrze RR.

Blok wykonania rozkazu zawiera jednostkę arytmetyczno-logiczną, która realizuje funkcje logiczne i arytmetyczne określone w polu ALU aktualnego mikrorozkazu[5]. W ramach jednostki arytmetyczno-logicznej występują dwa wejścia sumatora, zazwyczaj przedstawiane jako LALU i RALU, oraz wyjście jednostki. Podobnie jak w przypadku szyny BUS, wejścia sumatora nie przechowują wartości po wykonaniu taktu jednostki ALU.

Na rys. 4.3 zamieszczono schemat bloku wykonania rozkazu komputera LabZSK. Widoczne rejestry oraz magistrala wewnętrzna są połączone mikrooperacjami przesłań (ze wskazaniem kierunku przepływu).



Rys. 4.3. Blok wykonania rozkazu komputera LabZSK.

Flaga OFF jest zapalana przez wyjście ALU, jeśli wykryty został nadmiar:

- w operacji dodawania, operandy są tego samego znaku, a wynik przeciwnego znaku w stosunku do operandu,
- w operacji odejmowania, operandy są przeciwnego znaku, a wynik ma ten sam znak co lewy operand.

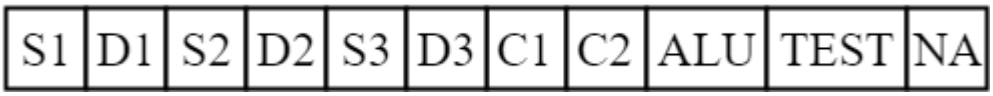
Flaga ZNAK jest zapalana (ustawiana), jeśli najstarszy bit (bit znaku) na wyjściu ALU jest ustawiony.

[5] Aktualnie wykonywany mikrorozkaz jest przechowywany w rejestrze RBPS.

5. Sterowanie mikroprogramowe

5.1. Organizacja pamięci stałej

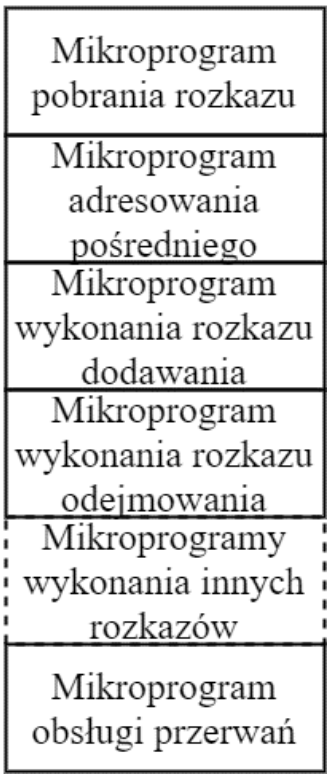
W komputerze LabZSK wykonaniu przez procesor podlegają sygnały sterujące zawarte w 48-bitowym słowie, w pamięci mikroprogramów. Każdy pobrany mikrorozkaz, jak przedstawiono na rys. 5.1, możemy podzielić na 10 mikrooperacji oraz pole z adresem następnego mikrorozkazu do wykonania (NA), przy spełnionym warunku skoku. Komputer LabZSK dysponuje 1,5 KB (256*48b) pamięcią stałą, przechowującą mikroprogramy składające się z mikrorozkazów, które z kolei zbudowane są z mikrooperacji. Zawartość pamięci stałej, czyli mikroprogramy, posłuży do realizacji rozkazów umieszczanych w 256 komórkach pamięci operacyjnej.



Rys. 5.1. Budowa mikrorozkazu komputera LabZSK.

Pamięć sterująca zbudowana jest z mikrorozkazów zawierających kody mikrooperacji, niezbędnych do wykonania. Sekwencja mikrooperacji w rozkazie jest ustalona. Dodatkowo każdy mikrorozkaz zawiera warunek skoku oraz adres następnego mikrorozkazu, realizowanego jeśli warunek jest spełniony; jeżeli warunek nie jest spełniony, wykonany będzie następny mikrorozkaz z pamięci stałej. Wykorzystując pamięć sterującą, procesor musi być w stanie pobierać i wykonywać kolejne rozkazy z pamięci operacyjnej. Wynika z tego, że w pamięci stałej muszą znaleźć się mikroprogramy pobrania, adresowania pośredniego, przerwania oraz wykonania dla każdej operacji; przy czym zawartość pamięci sterującej musi uwzględniać (w algorytmie), że cykle pobrania i wykonania występują zawsze dla każdego rozkazu, a adresowanie pośrednie i obsługa przerwań są opcjonalne.

Cykle pobrania, adresowania pośredniego i przerwania są proste i przewidywalne. Każdy z nich obejmuje niedużą ustaloną sekwencję mikrooperacji i w każdym przypadku wykonywane są te same mikrorozkazy. W pamięci stałej, jak przedstawiono na rys. 5.2, zapisany będzie tylko jeden mikroprogram dla cyklu pobrania, adresowania pośredniego oraz przerwania. Dla cyklu wykonania, konieczne jest dostarczenie mikroprogramów dla obsługi wszystkich instrukcji z listy rozkazów procesora. Skrajnie możliwe jest wystąpienie tylu sekwencji mikroprogramów, ile jest obsługiwanych rozkazów.



Rys. 5.2. Organizacja pamięci sterującej.

Mikroprogramy zawarte w pamięci sterowania składają się z mikrorozkazów. Każdy mikrorozkaz dzieli się na mikrooperacje, wykonywane w kolejnych taktach pracy procesora. Mikrooperacja angażuje odpowiednie zasoby procesora sygnałami sterującymi, jest prosta i odpowiada za elementarną operację.

5.2. Sygnały sterujące

Komputer LabZSK operuje na 48-bitowym słowie pobieranym z pamięci sterowania, będącym pełnym mikrorozkazem. Zawartość pamięci stałej, określona przez rejestr RAPS, jest pobierana przez procesor do rejestru RBPS. W strukturze każdego mikrorozkazu możemy wyodrębnić jedenaście części, zawierających sygnały dla procesora w określonych taktach jego pracy.

Kod	Pole S1	Definicja	Czas realizacji
1	IXRE	RI -> LALU	D1
2	OLR	LR -> BUS	D1
3	ORR	RR -> BUS	D1
4	ORAE	RAE -> BUS	D1
5	IALU	A -> LALU	D1
6	OXE	X -> RALU	D1
7	OX	X -> BUS	D1

Tabela 5.1. Sygnały sterujące kolumny S1.

Kod	Pole D1	Definicja	Czas realizacji
1	ILK	BUS -> LK	D1
2	IRAP	BUS -> RAP	D1
3	OXE	X -> RALU	D1

Tabela 5.2. Sygnały sterujące kolumny D1.

Kod	Pole S2	Definicja	Czas realizacji
1	IRAE	SUMA -> RAE	D6
2	ORR	RR -> BUS	D6
3	ORI	RI -> BUS	D6
4	ORAE	RAE -> BUS	D6
5	OA	A -> BUS	D6
6	OMQ	MQ -> BUS	D6
7	OX	X -> BUS	D6
8	OBE	ALU -> BUS	D6
9	IXRE	RI -> LALU	D6
10	IALU	A -> LALU	D6
11	OXE	X -> RALU	D6

Tabela 5.3. Sygnały sterujące kolumny S2.

Kod	Pole D2	Definicja	Czas realizacji
1	ILR	BUS -> LR	D6
2	IX	BUS -> X	D6
3	IBE	BUS -> RALU	D6
4	ORI	RI -> BUS	D6
5	IBI	BUS -> RAE	D6
6	IA	BUS -> A	D6
7	IMQ	BUS -> MQ	D6
8	OXE	X -> RALU	D6
9	NSI	LR+1 -> LR	D6
10	IAS	A_0 -> ZNAK	D6
11	SGN	X_0 -> ZNAK	D6
12	ALA	Arytmetyczne przesunięcie A w lewo	D1
13	ARA	Arytmetyczne przesunięcie A w prawo	D1
14	LRQ	Logiczne przesunięcie (A || MQ) w prawo	D1
15	LLQ	Logiczne przesunięcie (A || MQ) w lewo	D1
16	LLA	Logiczne przesunięcie A w lewo	D1
17	LRA	Logiczne przesunięcie A w prawo	D1
18	LCA	Cykliczne przesunięcie A w lewo	D1

Tabela 5.4. Sygnały sterujące kolumny D2.

Kod	Pole S3	Definicja	Czas realizacji
1	ORI	RI -> BUS	D7
2	OLR	LR -> BUS	D7
3	OA	A -> BUS	D7
4	ORAE	RAE -> BUS	D7
5	OMQ	MQ -> BUS	D7
6	ORBP	A -> LALU	D7
7	OXE	X -> RALU	D7

Tabela 5.5. Sygnały sterujące kolumny S3.

Kod	Pole D3	Definicja	Czas realizacji
1	ILR	BUS -> LR	D7
2	IX	BUS -> X	D7
3	IBE	BUS -> RALU	D7
4	IRI	BUS -> RI	D7
5	IBI	BUS -> RAE	D7
6	IA	BUS -> A	D7
7	IMQ	BUS -> MQ	D7
8	OXE	X -> RALU	D7
9	NSI	LR+1 -> LR	D7
10	IAS	A_0 -> ZNAK	D7
11	SGN	X_0 -> ZNAK	D7
12	IRI	BUS -> RR	D7
13	IRBP	BUS -> RBP	D7
14	SRBP	BUS -> RBP	D7

Tabela 5.6. Sygnały sterujące kolumny D3.

Kod	Pole C1	Definicja	Czas realizacji
1	CWC	Rozpocznij cykl zerowanie- zapis	D7
2	RRC	Rozpocznij cykl odczyt-regeneracja	D1
3	MUL	16 -> LK	D1
4	DIV	15 -> LK	D1
5	SHT	Operacja przesunięcia	D1
6	IWC	Rozpocznij cykl przerwaniowy	D7
7	END	Koniec mikroprogramu	D7

Tabela 5.7. Sygnały sterujące kolumny C1.

Kod	Pole C2	Definicja	Czas realizacji
1	DLK	LK = [LK]-1	D6
2	SOFF	OFF = 1	D6
3	ROFF	OFF = 0	D6
4	SXRO	XRO = 1	D6
5	RXRO	XRO = 0	D6
6	DRI	RI = RI-1	D6
7	RA	A = 0	D6
8	RMQ	MQ = 0	D6
9	AQ15	NOT A_0 -> MQ_{15}	D6
10	RINT	INT = 0	D7
11	OPC	OP lub AOP+32 -> RAPS	D7
12	CEA	Oblicz adres efektywny	D1
13	ENI	Odblokuj przerwania	D7

Tabela 5.8. Sygnały sterujące kolumny C2.

Kod	Pole TEST	Definicja	Czas realizacji
1	UNB	Zawsze pozytywny	D7
2	TINT	Brak przerwania	D7
3	TIND	Adresowanie pośrednie	D7
4	TAS	A >= 0	D7
5	TXS	RI >= 0	D7
6	TQ15	$MQ_{15} = 0$	D7
7	TLK	SHT, LK = 0 || !SHT, LK != 0	D7
8	TSD	ZNAK = 0	D7
9	TAO	OFF = 0	D7
10	TXP	RI <= 0	D7
11	TXZ	BXZ i RI != 0 || TLD i RI = 0	D7
12	TXRO	XRO = 0	D7
13	TAP	A <= 0	D7
14	TAZ	A = 0	D7

Tabela 5.9. Sygnały sterujące kolumny TEST.

Kod	Pole ALU	Definicja	Czas realizacji
1	ADD	ALU=LALU+RALU	D2-D5
2	SUB	ALU=LALU-RALU	D2-D5
3	CMX	ALU=NOT RALU+1	D2-D5
4	CMA	ALU=NOT LALU+1	D2-D5
5	OR	ALU=LALU OR RALU	D2-D5
6	AND	ALU=LALU AND RALU	D2-D5
7	EOR	ALU=LALU EOR RALU	D2-D5
8	NOTL	ALU=NOT LALU	D2-D5
9	NOTR	ALU=NOT RALU	D2-D5
10	L	ALU=LALU	D2-D5
11	R	ALU=RALU	D2-D5
12	INCL	ALU=LALU+1	D2-D5
13	INCR	ALU=RALU+1	D2-D5
14	DECL	ALU=LALU-1	D2-D5
15	DECR	ALU=RALU-1	D2-D5
16	ONE	ALU=1	D2-D5
17	ZERO	ALU=0	D2-D5

Tabela 5.10. Sygnały sterujące kolumny ALU.

5.3. Cykle pracy pamięci operacyjnej

RRC – cykl odczyt-regeneracja

W cyklu odczyt-regeneracja zawartość komórki PAO, adresowanej przez RAP, jest przesyłana szyną danych do rejestru RBP.

RBP=PAO[RAP]

CWC – cykl zerowanie-zapis

W trakcie cyklu zerowanie-zapis komórka pamięci operacyjnej, adresowanej przez RAP, jest zerowana a następnie ładowana zawartością rejestru RBP.

PAO[RAP]=0, PAO[RAP]=RBP

IWC – cykl przerwaniowy

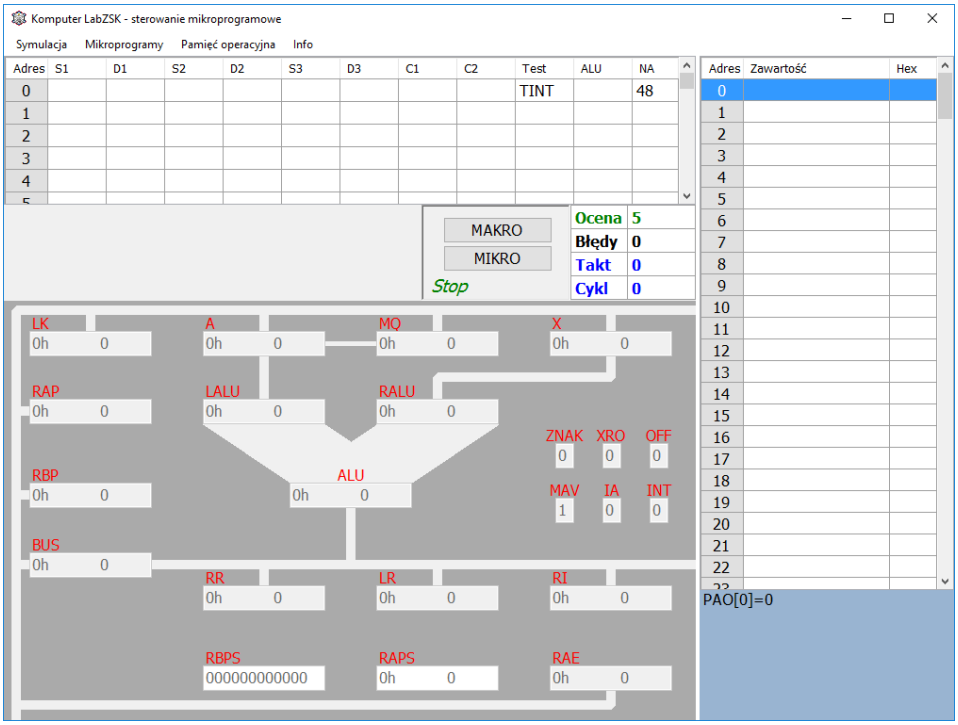
Zaistnienie cyklu przerwaniowego powoduje, że RAP jest sprzętowo ładowany wartością 255. Następnie zawartość licznika rozkazów (LR) trafia do komórki PAO wskazywanej przez RAP.

RAP=255, PAO[RAP]=LR

6. Graficzny interfejs symulatora

W skład interfejsu użytkownika wchodzą:

- edytor pamięci mikroprogramów,
- edytor pamięci operacyjnej,
- symulator komputera LabZSK,
- menu ustawień symulatora.



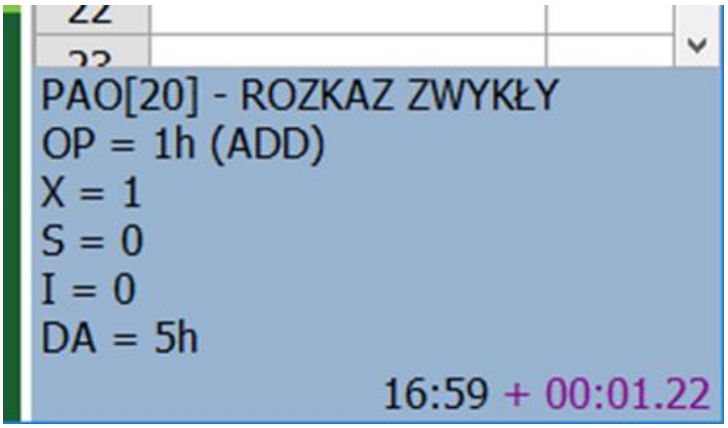
Rys. 6.1. Okno symulacji pracy procesora aplikacji LabZSK.

Uruchomienie symulatora prowadzi do wyświetlenia, przedstawionego na rys. 6.1, okna symulacji[6], będącego bazą do podejmowania dalszych działań oraz przechodzenia do innych funkcji systemu. Pasek menu pozwala na szybkie przejście do pozostałych widoków systemu lub wykonanie predefiniowanych akcji, bez konieczności wywoływania innych okien systemu. Wywoływanie kolejnych okien zazwyczaj nie blokuje dostępu do innych części aplikacji, wyjątkami są: okno

[6] Jeśli symulator został uruchomiony bez pliku konfiguracyjnego, wyświetlenie okna symulacji zostanie poprzedzone odpowiednim komunikatem.

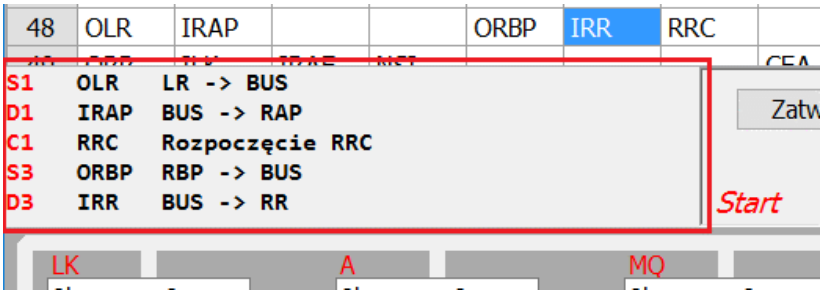
ustawień programu, okno wyświetlania zawartości pliku logu oraz okno informacji o autorze. Pozostałe okna pozwalają na przełączanie się między nimi w dowolnym momencie pracy.

Okno symulatora posiada wbudowany podgląd zawartości pamięci stałej oraz operacyjnej. Sama pamięć operacyjna może być również modyfikowana z poziomu tego okna, bez konieczności wywołania edytora. Wystarczy rozpocząć wprowadzanie danych uprzednio zaznaczając interesującą nas pozycję w pamięci. Dodatkowo wybranie dowolnej komórki z pamięci pozwala na rozkodowanie jej zawartości w oknie, znajdującym się w prawym dolnym rogu symulatora. Rozkodowanie rozkazów i danych, jak widać na rys. 6.2, jest uproszczone względem głównego okna pamięci operacyjnej, z racji dostępnej przestrzeni.



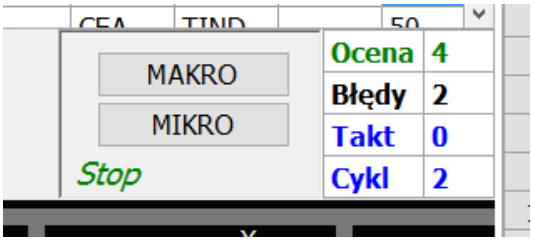
Rys. 6.2. Fragment okna symulacji – obszar rozkodowania rozkazów.

Pod widokiem pamięci stałej, umieszczonej przy górnej krawędzi okna, co zobrazowano na rys. 6.3, widnieje obszar na skrócone informacje z dziennika aktywności. Pozwala on na szybkie odnalezienie poprzednich oraz aktualnie wykonywanych mikrooperacji. Obszar ten, po wykonaniu pełnego cyklu rozkazowego, zostaje wyczyszczony i zastąpiony przyciskiem wyświetlenia pełnego logu z pracy.



Rys. 6.3. Fragment okna symulacji – skrócony dziennik pracy.

W głównym oknie programu widoczne są również przyciski wyboru trybu symulacji i podglądu stanu symulacji i symulatora. W obszarze statystyk możemy uzyskać informacje na temat aktualnej liczby popełnionych błędów, bieżącego cyklu oraz taktu pracy procesora. Lewy dolny róg zarezerwowany jest dla widoku pracy procesora, zapewniając możliwość podglądu oraz edycji rejestrów i flag. W oknie symulacji dostępne są również przyciski wyboru trybu pracy procesora. Wybranie któregokolwiek spowoduje zmianę stanu procesora ze „Stop” na „Start” oraz przygotowanie do dalszego procesu symulacji, zgodnie z wybranym trybem pracy.



Rys. 6.4. Fragment okna symulacji – tryby pracy i podgląd stanu.

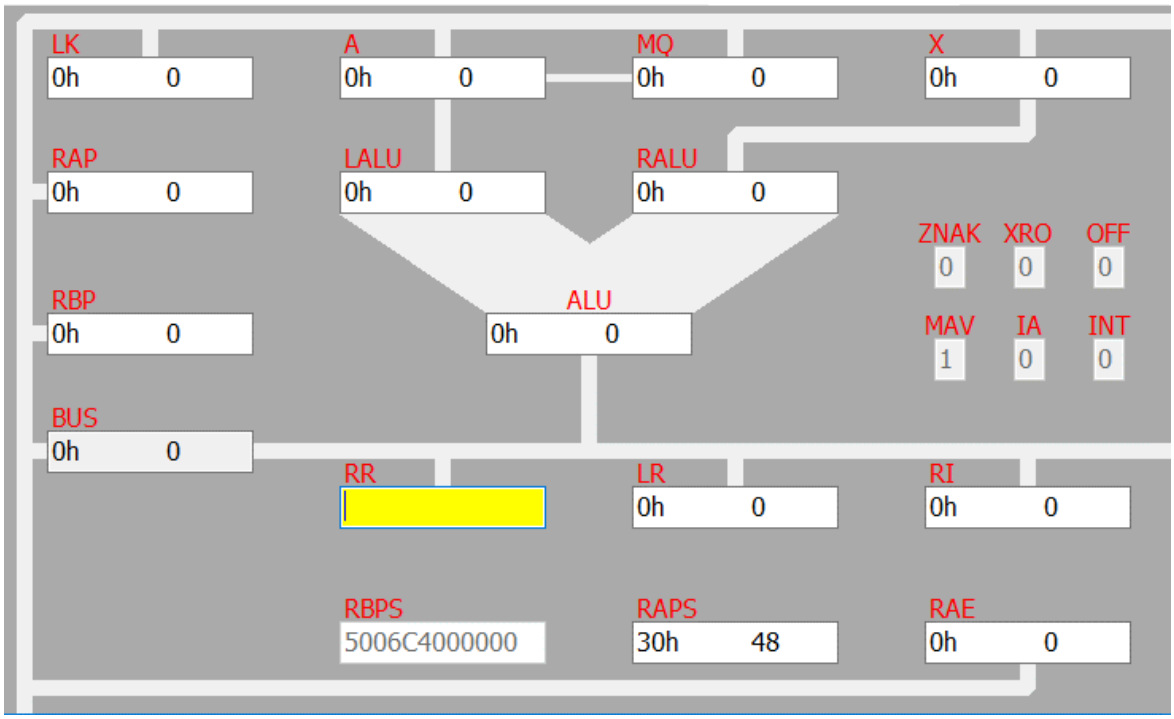
Edytowanie rejestrów procesora odbywa się poprzez wykorzystanie paska menu lub dwukliku myszy w obszarze zobrazowania jednostki sterującej. Wejście w tryb edycji zostanie odpowiednio zakomunikowane poprzez zmianę aktualnego

stanu procesora. Tryb edycji pozwala na zmianę wartości większości rejestrów poza rejestrem RBPS, wejściami jednostki ALU oraz stanu na szynie BUS. Tryb edycji powinien być stosowany przed właściwym rozpoczęciem pracy, aby wstępnie ustawić żądane parametry w procesorze komputera. Edycja zawartości rejestrów zostanie odpowiednio odnotowana w dzienniku symulacji, jeśli nie zostały one zmodyfikowane przed pierwszym startem jednostki sterującej.

Wykorzystując pasek menu, zlokalizowany w lewym górnym rogu okna, możliwe jest wyzerowanie zawartości rejestrów oraz flag. Ponadto dostępne są też pozycje odpowiedzialne za wyświetlenie zawartości wskazanego dziennika przeprowadzonej symulacji lub przejście do menu ustawień programu.

Wspominany dzienniki pracy jest niezbędne do uruchomienia symulacji, w ramach której zapisywane będą wszelkie szczegóły z pracy procesora. Zamknięcie dziennika i podpisanie go sumą kontrolną zazwyczaj odbywa się przy zamykaniu aplikacji symulatora.

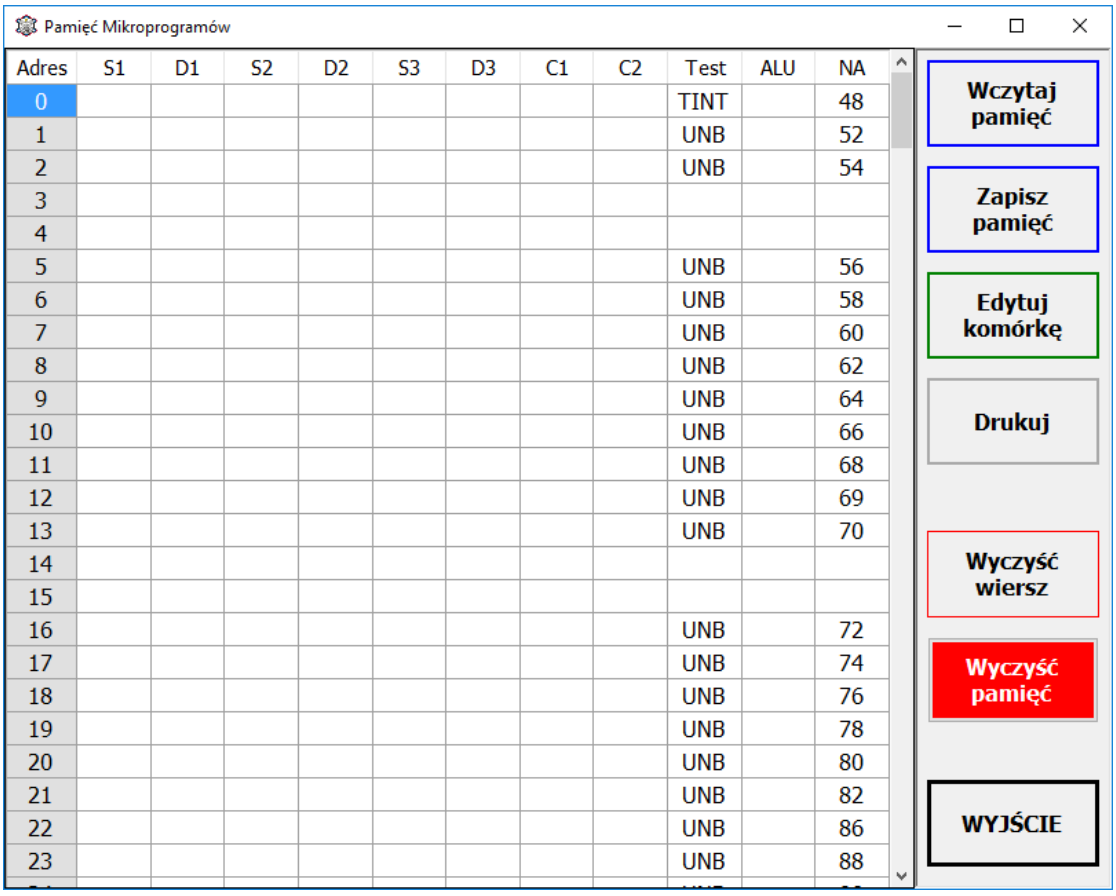
Pole oznaczane jaskrawym żółtym tłem, widocznym na rys. 6.5, wskazuje rejestr, którego wartość należy zaktualizować. Nową wartość rejestru można wprowadzić za pomocą klawiatury lub przeciągając myszką zawartość innego rejestru. Wprowadzane wartości będą domyślnie traktowane jako liczby dziesiętne. Wprowadzenie wartości zakończonej literą „h”, wymusza tryb heksadecymalny. Jeśli wprowadzony zostanie ciąg zawierający elementy charakterystyczne dla liczby heksadecymalnej (litery „ABCDEF”) domyślnie zostanie potraktowany jako liczba heksadecymalna.



Rys. 6.5. Fragment okna symulacji – rejestr oznaczony do wprowadzania.

Wykorzystując pasek menu możliwe jest wywołanie okna edycji pamięci stałej, widocznego na rys. 6.6, używanego do tworzenia mikroprogramów wykonywanych później w symulatorze. Okno edycji umożliwia także zapisywanie, wczytywanie oraz drukowanie zawartości pamięci stałej do pliku tekstowego. Wszelkie zmiany w oknie pamięci mikroprogramów są automatycznie widoczne w głównym oknie pracy komputera.

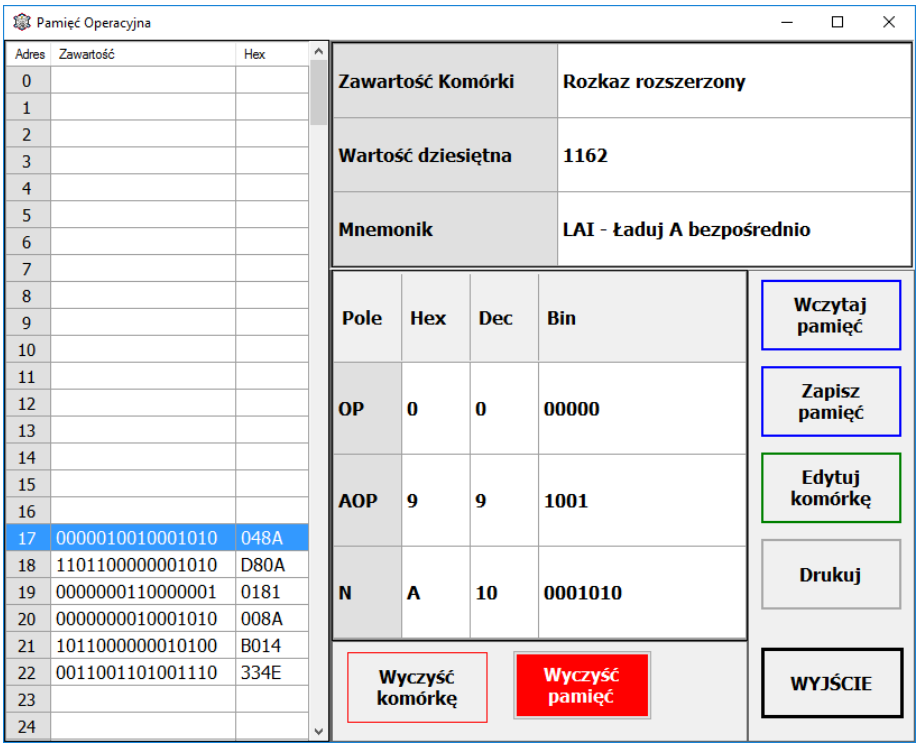
Wprowadzenie mikrooperacji dla wskazanej kolumny odbywa się poprzez przywołanie okna, proponującego wybranie instrukcji z odpowiedniego zestawu. Przywołanie takiego okna może odbyć się poprzez dwuklik myszy na komórce lub wykorzystanie przycisku „Edytuj komórkę". Skorzystanie z przycisku należy poprzedzić zaznaczeniem komórki, której zawartość ma być modyfikowana.



Adres	S1	D1	S2	D2	S3	D3	C1	C2	Test	ALU	NA
0									TINT		48
1									UNB		52
2									UNB		54
3											
4											
5									UNB		56
6									UNB		58
7									UNB		60
8									UNB		62
9									UNB		64
10									UNB		66
11									UNB		68
12									UNB		69
13									UNB		70
14											
15											
16									UNB		72
17									UNB		74
18									UNB		76
19									UNB		78
20									UNB		80
21									UNB		82
22									UNB		86
23									UNB		88

Rys. 6.6. Okno edycji pamięci mikroprogramów aplikacji LabZSK.

Praca z edytorem pamięci operacyjnej, przedstawionym na rys. 6.7, jest możliwa po wcześniejszym wywołaniu go z paska menu. W ramach pracy z edytorem, użytkownik otrzymuje możliwość wprowadzenia dowolnej wartości, mieszczącej się w 16-bitowych komórkach pamięci. Rozróżnienie typów zawartości stosowane jest dla usprawnienia procesu wypełniania pamięci operacyjnej. Komputer sterowany mikroprogramowo nie posiada jakiejkolwiek informacji o typach zawartości komórek pamięci operacyjnej. Zawartość dowolnej komórki może być traktowana jako dana, adres albo pobierana jako rozkaz. Rozróżnienie typów danych jest stosowane dla wygody użytkowania przez człowieka.



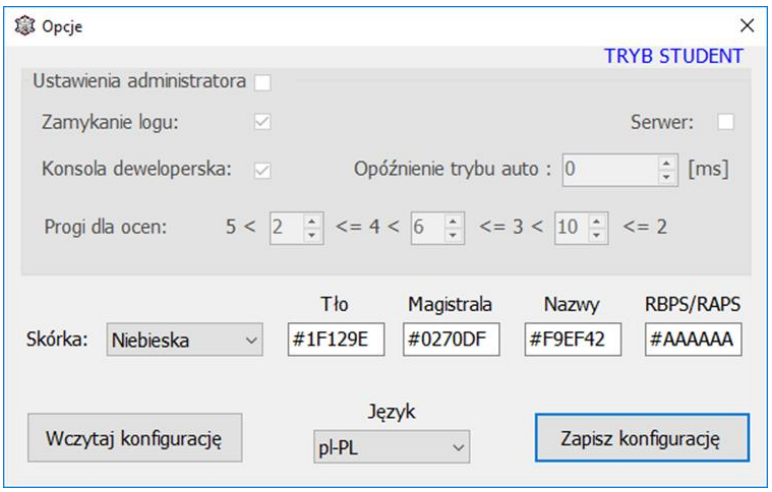
Rys. 6.7. Okno edycji pamięci operacyjnej aplikacji LabZSK.

Wykorzystanie wbudowanego edytora pozwala na konstruowanie rozkazów zwykłych i rozszerzonych o zadanych polach. Dozwolone jest bezpośrednie wprowadzanie danych liczbowych inicjowane klawiaturą, a alternatywnie możliwe jest wywołanie odpowiedniego okna przyciskiem „Edytuj komórkę”.

Wybranie dowolnej niepustej komórki spowoduje rozkodowanie jej zawartości, w dwóch tabelach. Pierwsza tabela przedstawi wybrany typ oraz wartość liczbową komórki. Zależnie od typu, zostanie przedstawiona nazwa mnemoniczna oraz krótki

opis (pod warunkiem, że był wprowadzony jako rozkaz). W drugiej tabeli niezależnie od abstrakcyjnego typu komórki wybranej przez użytkownika, zawartość zostanie rozkodowana jako rozkaz.

Konfiguracja ustawień symulatora jest dostępna po wywołaniu okna ustawień. Przywołane opcje aplikacji, widoczne na rys. 6.8, pozwalają na modyfikowanie udostępnionych parametrów programu. Same opcje podzielone zostały również na ogólnodostępne i zastrzeżone dla administratora. Do pierwszej kategorii zaliczyć należy możliwość wywołania serwera, zmianę języka bądź skórki symulatora. Prócz predefiniowanych motywów, udostępniono również możliwość zadawania własnych wartości i korzystania z nich przy kolejnych uruchomieniach programu. Pozostałe opcje takie jak, możliwość zamykania logu, otwierania konsoli deweloperskiej czy modyfikowanie progów oceniania zostały zablokowane dla użytkownika bez odpowiednich uprawnień.



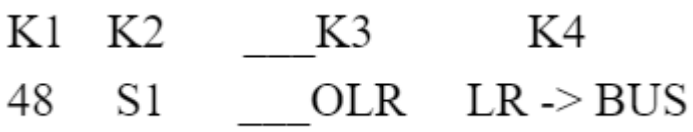
Rys. 6.8. Okno ustawień aplikacji LabZSK (w trybie STUDENT).

Wykorzystując przycisk „Zapisz konfigurację” aktualna kombinacja ustawień z menu opcji zostanie zapisana w specjalnym pliku konfiguracyjnym. Dostęp do ustawień administratora uzyskać można tylko wczytując konfigurację podnoszącą uprawnienia aplikacji. Wczytywanie pliku zapisanej konfiguracji należy inicjować przyciskiem „Wczytaj konfigurację” znajdującym się w tym samym oknie.

Zmiany wykonane w oknie ustawień są automatycznie zapisywane do aktualnej konfiguracji i nie trzeba ich ręcznie zapisywać.

7. Przykładowy mikroprogram pobrania rozkazu

Działanie mikroprogramu pobrania (oraz wykonania rozkazów ADD i SUB), zostanie opisane na podstawie zawartości wydruku z pamięci stałej.



Rys. 7.1. Pojedynczy wiersz z wydruku zawartości pamięci stałej – numerowane kolumny mikrooperacji.

Kolejne pola wydruku dla mikrooperacji, z rys. 7.1, należy interpretować następująco:

- Liczba w kolumnie K1 oznacza adres komórki pamięci stałej, której zawartość podlega opisowi (jeśli kolumna K1 jest pusta oznacza to, że kontynuowany jest poprzedni adres).
- Symbol w kolumnie K2 oznacza pole mikrooperacji.
- Symbol z kolumny K3 przedstawia mnemonik realizowanej mikrooperacji.
- Wzór lub tekst z kolumny K4 opisuje rodzaj działania dla powyższej mikrooperacji.

Symbolem „#” oznaczono linie z komentarzami do zawartości pamięci (nie są one częścią wydruku).

Wydruk zawartości pamięci stałej

```
# sprawdzenie wystąpienia przerwania

0       Test    ___TINT    Brak przerwania

        NA      ___48

# skok do mikroprogramu rozkazu ADD

1       Test    ___UNB     Zawsze pozytywny

        NA      ___52

# skok do mikroprogramu realizacji rozkazu SUB

2       Test    ___UNB     Zawsze pozytywny
```

```
        NA      ___54
# mikroprogram pobrania rozkazu
48      S1      ___OLR     LR -> BUS
        D1      ___IRAP    BUS -> RAP
        S3      ___ORBP    RBP -> BUS
        D3      ___IRR     BUS -> RR
        C1      ___RRC     Rozpoczęcie RRC
49      S1      ___ORR     RR -> BUS
        D1      ___ILK     BUS -> LK
        S2      ___IRAE    SUMA -> RAE
        D2      ___NSI     LR+1 -> LR
        C2      ___CEA     Oblicz adres efektywny
        Test    ___TIND    Adresowanie pośrednie
        NA      ___50
# mikroprogram adresowania pośredniego
50      S1      ___ORAE    RAE -> BUS
        D1      ___IRAP    BUS -> RAP
        S3      ___ORBP    RBP -> BUS
        D3      ___IX      BUS -> X
        C1      ___RRC     Rozpoczęcie RRC
51      S2      ___OX      X -> BUS
        D2      ___IBI     BUS -> RAE
        C2      ___OPC     OP lub AOP+32 -> RAPS
# mikroprogram wykonania rozkazu ADD
52      S1      ___ORAE    RAE -> BUS
        D1      ___IRAP    BUS -> RAP
        S3      ___ORBP    RBP -> BUS
        D3      ___IX      BUS -> X
        C1      ___RRC     Rozpoczęcie RRC
```

```
53      S1      ___IALU    A -> LALU
        D1      ___OXE     X -> RALU
        S2      ___OBE     ALU -> BUS
        D2      ___IA      BUS -> A
        C1      ___END     Koniec mikroprogramu
        ALU     ___ADD     ALU = LALU + RALU
# mikroprogram wykonania rozkazu SUB
54      S1      ___ORAE    RAE -> BUS
        D1      ___IRAP    BUS -> RAP
        S3      ___ORBP    RBP -> BUS
        D3      ___IX      BUS -> X
        C1      ___RRC     Rozpoczęcie RRC
55      S1      ___IALU    A -> LALU
        D1      ___OXE     X -> RALU
        S2      ___OBE     ALU -> BUS
        D2      ___IA      BUS -> A
        C1      ___END     Koniec mikroprogramu
        ALU     ___SUB     ALU = LALU - RALU
```

Przedstawienie działania mikroprogramu

Jednostka sterująca rozpoczyna pracę od pobrania i wykonania mikrorozkazu z komórki pamięci stałej o adresie 0. Pobrany mikrorozkaz zawiera test TINT, sprawdzający wystąpienie przerwania. Jeśli wystąpiło przerwanie, sprzętowo ładowana jest wartość 255 do rejestru RAP, oraz 254 do rejestru RAPS. Celem przejścia do podprogramu obsługi przerwań (umieszczonego w komórce 254 pamięci stałej). Jeśli nie wystąpiło przerwanie (dla testu TINT) sterowanie jest przekazywane do komórki wskazywanej przez kolumnę NA, wykonywanego mikrorozkazu (w tym wypadku 48).

Zatem, w 48 komórce pamięci sterującej zawarty jest początek mikroprogramu pobrania rozkazu. Wykonanie mikrooperacji OLR oraz IRAP, powoduje przesłanie aktualnej zawartości LR do RAP. Następnie wykonywana jest mikrooperacja RRC, pobierająca z pamięci operacyjnej zawartość komórki adresowanej przez, załadowany wcześniej zawartością LR, rejestr RAP. Zawartość komórki PAO jest przesyłana szyną danych do RBP, a następnie za pomocą operacji ORBP i IRR przesyłana do rejestru RR. Ponieważ pole TEST jest puste, sterowanie jest przekazywane do następnej komórki w pamięci stałej.

Kolejny mikrorozkaz (z komórki 49) za pomocą mikrooperacji ORR i ILK powoduje przesłanie zawartości rejestru RR do LK. Skutkuje to załadowaniem operandu rozkazu (pole DA dla rozkazu zwykłego, pole N dla rozkazu rozszerzonego) do LK. Ponieważ rejestr LK jest 7-bitowy, dla rozkazów zwykłych, starsza część słowa bitowego jest ucinana. Następnie wykonywana jest mikrooperacja CEA, wyznaczająca adres efektywny argumentu. W wyniku tej mikrooperacji, operand rozkazu (zawartego w RR) jest ładowany na lewe wejście sumatora adresowego. Prawe wejście sumatora zostanie załadowane zgodnie z aktualnym trybem adresacji (lub zerem w przypadku rozkazu rozszerzonego). Kolejna mikrooperacja, czyli IRAE, ładuje obliczony adres do RAE. Następnie realizacja mikrooperacji NSI powoduje zwiększenie zawartości rejestru licznika rozkazów LR o 1, celem wskazania na następny rozkaz do wykonania. Ostatnią mikrooperacją w sekwencji jest test TIND, sprawdzający wystąpienie adresacji pośredniej. Jeśli test TIND wykazał adresowanie pośrednie (lub pobrano rozkaz rozszerzony), do RAPS ładowany jest adres wskazywany przez pole NA mikrorozkazu (w tym wypadku 50). W przeciwnym wypadku, sterowanie przekazywane jest do komórki o adresie równym zawartości pola OP aktualnie wykonywanego rozkazu.

Jeśli wystąpiło adresowanie pośrednie (lub pobrano rozkaz rozszerzony) i w wyniku testu TIND, sterowanie zostało przekazane do 50 komórki pamięci stałej. Mikrooperacjami ORAE oraz IRAP, aktualna zawartość RAE jest przesyłana do rejestru RAP. Następnie wykonywana jest mikrooperacja RRC, powodująca pobranie

zawartości komórki pamięci operacyjnej wskazywanej przez RAP do rejestru RBP. Pobrany argument jest następnie przesyłany do rejestru X, mikrooperacjami ORBP oraz IX. Ponieważ pole TEST jest puste, załadowany zostanie następny mikrorozkaz z sekwencji.

W 51 komórce pamięci stałej, zawarta jest druga część mikroprogramu adresowania pośredniego. Pobrana w poprzednim mikrorozkazie zawartość pamięci operacyjnej (i przeniesiona do rejestru X), mikrooperacjami OX oraz IBI jest przenoszona do RAE. Ostatnia mikrooperacja w sekwencji, czyli OPC, powoduje przekazanie sterowania do odpowiedniej komórki pamięci stałej. W przypadku rozkazów zwykłych, sterowanie jest przekazywane do komórki o adresie równym polu OP rozkazu, dla rozkazów rozszerzonych RAPS ładowany jest wartością pola AOP, zwiększoną o 32.

Realizacja rozkazu ADD:

Jeśli z pamięci został pobrany rozkaz ADD, RAPS zostanie załadowany wartością 1 (po mikrooperacji CEA z komórki 49 lub dla adresowania pośredniego po OPC z komórki 51). W 1 komórce pamięci stałej zawarty jest skoku bezwarunkowy do mikroprogramu obsługującego rozkaz ADD. Wykonanie tego skoku powoduje przekazywane sterowanie do 52 komórki pamięci stałej.

Rozpoczynając obsługę rozkazu ADD, w wyniku mikrooperacji ORAE i IRAP, zawartość rejestru RAE jest przesyłany do RAP. Następnie mikrooperacja RRC powoduje załadowanie do RBP wartości komórki pamięci operacyjnej adresowanej przez RAP. Wartość pobrana z pamięci jest następnie przesyłana do rejestru X mikrooperacjami ORBP i IX. Ponieważ pole Test jest puste, sterowanie przekazywane jest do następnej komórki pamięci stałej.

W 53 komórce pamięci stałej zawarto mikrooperacje IALU i OXE, przenoszące argumenty z rejestrów A i X do odpowiednio LALU i RALU. Które są następnie sumowane przez jednostkę arytmetyczno-logiczną mikrooperacją ADD z kolumny ALU. Wynik operacji jest przesyłany z ALU do akumulatora A za pomocą mikrooperacji OBE i IA. Mikrorozkaz z 53 komórki pamięci stałej jest kończony

wykonaniem mikrooperacji END, przekazującej sterowanie do zerowej komórki pamięci stałej (powrót do sprawdzenia wystąpienia przerwania).

Realizacja rozkazu SUB:

Realizacja rozkazu SUB jest prawie identyczna jak rozkazu ADD. Wykonanie identycznego mikroprogramu pobrania rozkazu, przekazuje sterowanie do odpowiedniego mikroprogramu skokiem bezwarunkowym UNB.

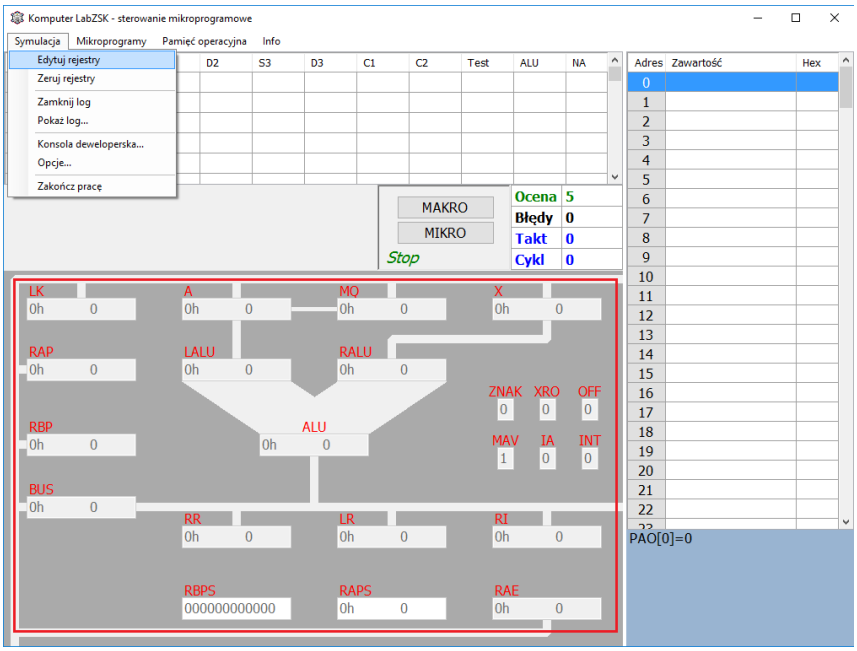
Pierwsza część mikroprogramu realizacji rozkazu SUB jest identyczna, występują w niej mikrooperacje ORAE, IRAP, RRC, ORBP i IX. W ramach drugiej części mikroprogramu, operacje przesłań są jednakowe a jedyną różnicą w sekwencji jest realizacja sygnału SUB, z kolumny ALU mikrorozkazu. Identycznie jak w przypadku ADD (oraz innych rozkazów), obsługa rozkazu SUB kończy się mikrooperacją END, ładującą RAPS wartością 0.

8. Rozpoczęcie pobrania rozkazu w symulatorze LabZSK

Pobranie rozkazu z pamięci, prezentowane jest z wykorzystaniem mikroprogramu z poprzedniego rozdziału (wykonania rozkazów ADS i SUB nie są zawarte).

Prezentacja procesu pobrania rozkazu z pamięci operacyjnej, została wykonana po uprzednim ustawieniu wartości rejestrów LR i RI na odpowiednio 14h i Ah.

Do trybu edycji zawartości rejestrów można przejść z paska menu lub stosując dwuklik myszy na obszarze procesora, widocznym na rys. 8.2 .



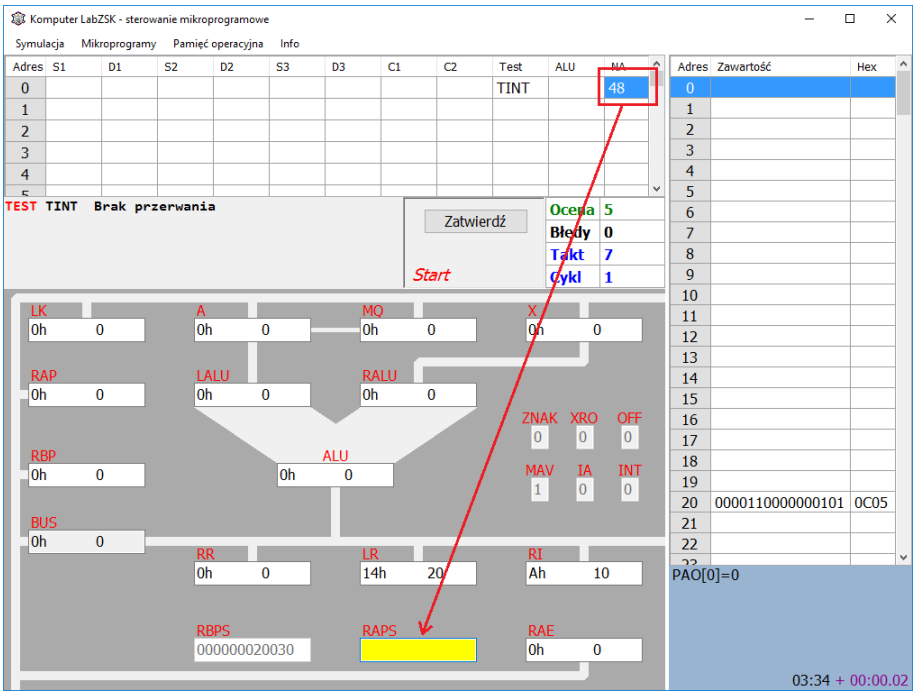
Rys. 8.2. Uruchomienie trybu edycji - na czerwono zaznaczono obszar procesora.

W trybie edycji zawartości rejestrów wszystkie liczby złożone z cyfr 0-9 są domyślnie traktowane jako dziesiętne. Wprowadzenie liter charakterystycznych dla wartości szesnastkowych (A-F) lub zakończenie dowolnej sekwencji literą „H”, wymusza tryb heksadecymalny.

Uruchomienie symulacji, z mikroprogramem pobrania rozkazu z pamięci głównej, spowoduje załadowanie mikrorozkazu z obszaru pamięci stałej, wskazywanego przez RAPS (w tym przypadku RAPS=0).

Załadowany mikrorozkaz rozpoczyna się testem TINT, sprawdzającym czy do procesora zostało zgłoszone przerwanie. Jeśli flaga INT jest zapalona (INT=1), mikrooperacja TINT spowoduje sprzętowe załadowanie wartości 255 na RAP oraz przekazanie sterowania do mikroprogramu obsługi przerwań (RAPS=254). W prezentowanym przykładzie (INT=0) test TINT zwróci wartość fałsz, oznacza to przekazanie sterowania do komórki pamięci stałej określonej przez pole NA, bieżącego mikrorozkazu.

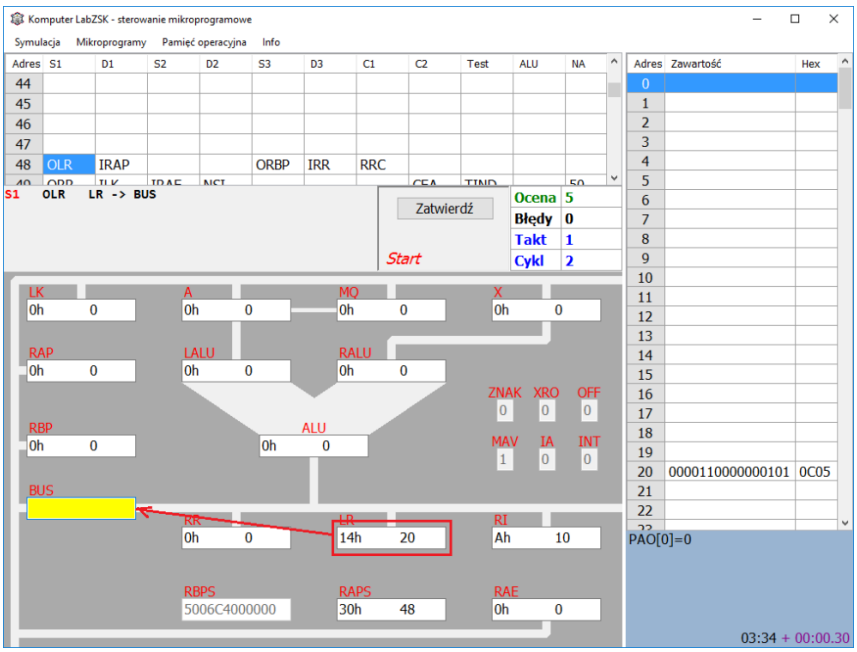
Jaskrawe tło rejestru, widoczne na rys. 8.3, sygnalizuje konieczność ręcznego wprowadzenia zaktualizowanej wartości. Wartość z pola NA (w prezentowanym przykładzie NA=48) należy ręcznie wprowadzić do RAPS.



Rys. 8.3. Etap aktualizacja wartości RAPS podczas testu TIND.

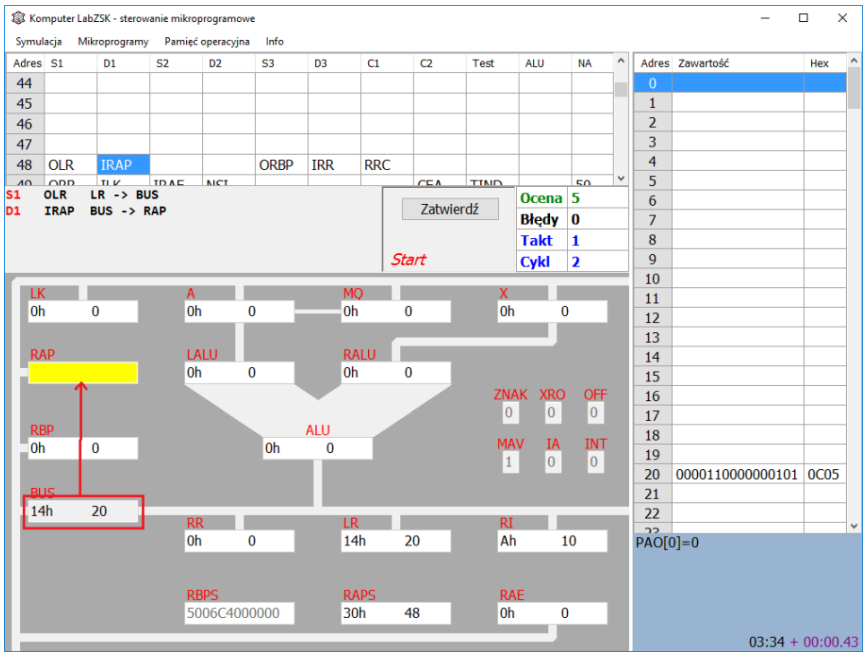
Kolejny mikrorozkaz do wykonania zostanie pobrany z komórki pamięci stałej wskazywanej przez RAPS (w prezentowanym przykładzie RAPS=48).

Nowy mikrorozkaz, pobrany z 48 komórki pamięci stałej, rozpoczną się od mikrooperacji OLR. Zgodnie z jej opisem (LR -> BUS), zawartość rejestru LR musi trafić na szynę BUS.



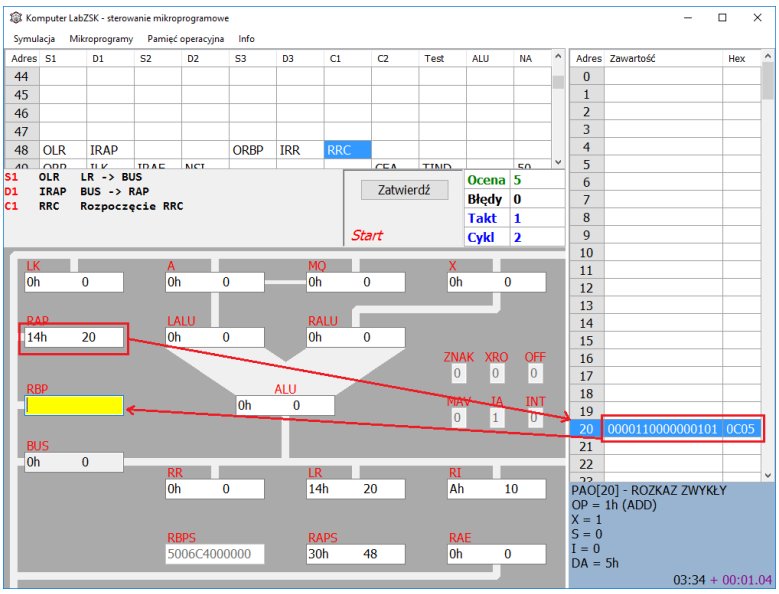
Rys. 8.4. Etap aktualizacji wartości BUS dla mikrooperacji OLR.

Następnie wartość umieszczona na szynie BUS, za pośrednictwem mikrooperacji IRAP, zostaje przesłana do rejestru adresacji pamięci (RAP).



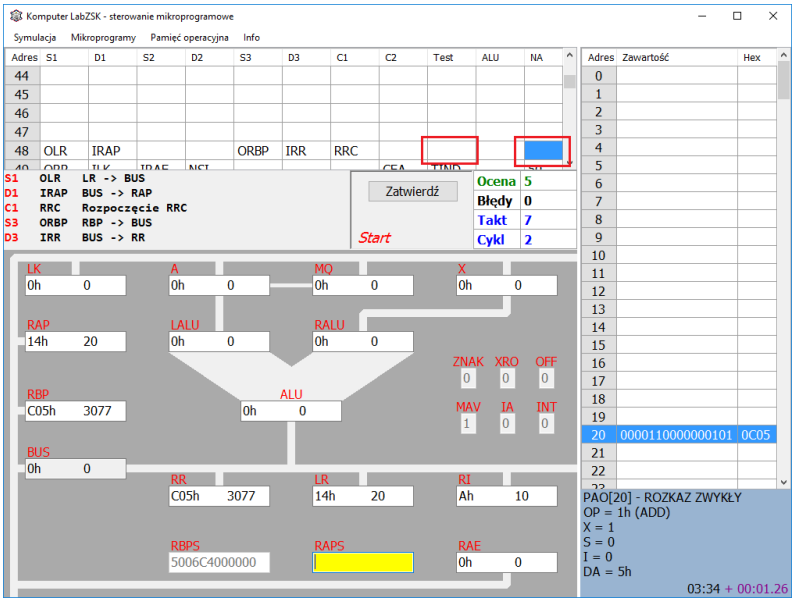
Rys. 8.5. Etap aktualizacji wartości rejestru RAP dla mikrooperacji IRAP.

Kolejna mikrooperacja w sekwencji, RRC, powoduje załadowanie rejestru buforowego pamięci RBP, wartością z komórki pamięci operacyjnej wskazywanej przez rejestr adresacji pamięci RAP (RBP=PAO[RAP]).



Rys. 8.6. Etap wykonania mikrooperacji RRC.

Mikrorozkaz w którym pola Test i NA pozostają puste, należy zakończyć wskazaniem następnej komórki z pamięci stałej. Oznacza to iż rejestr adresacji pamięci stałej musi zostać załadowany adresem kolejnego mikrorozkazu z pamięci mikroprogramów (zwiększamy RAPS o 1).



Rys. 8.7. Etap wprowadzenia adresu następnego mikrorozkazu do RAPS.

Wykaz tabel

Tabela 2.1. Rejestry komputera LabZSK (Alfabetycznie). 4
Tabela 3.1. Rozkazy komputera LabZSK w formacie zwykłym – cz. 1. 5
Tabela 3.2. Rozkazy komputera LabZSK w formacie zwykłym – cz. 2. 6
Tabela 3.3. Rozkazy w formacie rozszerzonym. 7
Tabela 3.4. Rozkazy arytmetyczne komputera LabZSK. 8
Tabela 3.5. Rozkazy logiczne komputera LabZSK. 8
Tabela 3.6. Rozkazy przesłań komputera LabZSK. 9
Tabela 3.7. Rozkazy skoku komputera LabZSK. 9
Tabela 3.8. Rozkazy przesunięć komputera LabZSK. 10
Tabela 3.9. Metody adresacji komputera LabZSK. 13
Tabela 5.1. Sygnały sterujące kolumny S1. 19
Tabela 5.2. Sygnały sterujące kolumny D1. 19
Tabela 5.3. Sygnały sterujące kolumny S2. 20
Tabela 5.4. Sygnały sterujące kolumny D2. 20
Tabela 5.5. Sygnały sterujące kolumny S3. 21
Tabela 5.6. Sygnały sterujące kolumny D3. 21
Tabela 5.7. Sygnały sterujące kolumny C1. 22
Tabela 5.8. Sygnały sterujące kolumny C2. 22
Tabela 5.9. Sygnały sterujące kolumny TEST. 23
Tabela 5.10. Sygnały sterujące kolumny ALU. 23

Wykaz rysunków

Rys. 3.1. Algorytm cyklu rozkazowy. 11
Rys. 3.2. Format rozkazu zwykłego komputera LabZSK. 12
Rys. 3.3. Format rozkazu rozszerzonego komputera LabZSK. 12
Rys. 4.1. Budowa jednostki centralnej. 14
Rys. 4.2. Blok pobrania rozkazu komputera LabZSK.....15
Rys. 4.3. Blok wykonania rozkazu komputera LabZSK. 16
Rys. 5.1. Budowa mikrorozkazu komputera LabZSK. 17
Rys. 5.2. Organizacja pamięci sterującej.....18
Rys. 6.1. Okno symulacji pracy procesora aplikacji LabZSK. 25
Rys. 6.2. Fragment okna symulacji – obszar rozkodowania rozkazów. 26
Rys. 6.3. Fragment okna symulacji – skrócony dziennik pracy.....27
Rys. 6.4. Fragment okna symulacji – tryby pracy i podgląd stanu. 27
Rys. 6.5. Fragment okna symulacji – rejestr oznaczony do wprowadzania. 29
Rys. 6.6. Okno edycji pamięci mikroprogramów aplikacji LabZSK.....30
Rys. 6.7. Okno edycji pamięci operacyjnej aplikacji LabZSK. 31
Rys. 6.8. Okno ustawień aplikacji LabZSK (w trybie STUDENT).....32
Rys. 7.1. Pojedynczy wiersz z wydruku zawartości pamięci stałej – numerowane kolumny mikrooperacji. 33
Rys. 8.2. Uruchomienie trybu edycji - na czerwono zaznaczono obszar procesora. 39
Rys. 8.3. Etap aktualizacja wartości RAPS podczas testu TIND.....40
Rys. 8.4. Etap aktualizacji wartości BUS dla mikrooperacji OLR. 41
Rys. 8.5. Etap aktualizacji wartości rejestru RAP dla mikrooperacji IRAP.....41
Rys. 8.6. Etap wykonania mikrooperacji RRC.....42
Rys. 8.7. Etap wprowadzenia adresu następnego mikrorozkazu do RAPS. 42